Exercise - Routing / Finishing

About the style

We will use a number of different styles to identify different types of actions as shown below:

Student Task

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

Note: *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex09
sh > cd ex09
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex09/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex09
```

Student Task 0

Additionally, we need to initialize the croc directory with the cockpit tool to set up all technology related files:

```
sh> cd croc
sh> icdesign ihp13 -update all -nogui
```

Introduction

In the previous exercises, you explored the fundamentals of the OpenROAD flow, including Exercise - Floorplanning to define the layout of the design and the macros, Exercise - Placement / Timing for the placement of the standard cells, and Exercise - Understanding clock tree to distribute the clock across the design. In the last we will perform the **Routing** of the design to connect the standard cells and macros, and then perform the **Finishing** steps to prepare the design for manufacturing.

In this exercise, we will focus on the critical steps of Routing and Finishing, which finalize the physical implementation of the design and prepare it for manufacturing. Specifically:

- **Routing** connects the standard cells and macros based on the netlist.
- **Finishing** ensures the design adheres to manufacturing requirements and is ready for tape-out.

Routing

Routing is a crucial stage in the physical design flow. It involves:

1. **Global Routing**: Planning approximate routes across chip regions to minimize congestion and guide the detailed routing stage.
2. **Optimize Routing**: Applying additional optimization steps to resolve timing issues, such as setup or hold violations, and to ensure the design meets performance goals.
3. **Detailed Routing**: Finalizing the exact interconnections while adhering to design rules and constraints.

Finishing

Once routing is complete, the design undergoes Finishing, which includes:

1. **Filler Cells**: Inserting additional filler cells in areas where standard cells are not placed, ensuring continuous power rails and preventing open areas.
2. **Metal Fill**: Adding metal fill patterns to meet manufacturing density requirements.

Finally, the outcome of this exercise will be DEF file, which represents the layout of all the cells and nets in your design. The DEF file can then be converted to a GDS file, the standard format used to represent the final design layout. Eventually, the GDS file will be sent for manufacturing, but first, Design Rule Checking (DRC) and Layout Versus Schematic (LVS) need to be performed, which will be covered in a subsequent exercise. By the end of this exercise, you will:

- Understand all the steps involved in the routing flow and what they entail.
- Learn how to analyze routing results and identify and resolve potential issues, such as timing violations and congestion.
- Perform the final finishing steps, including routing optimization and DEF generation, to complete the design.

Part 1: Routing resources

Before we start routing, we should be familiar with the metal stack, as it plays a significant role in the routing step since it determines the routing resources we have to route our design. We already had a first look at the metal stack in Exercise - Introduction to OpenRoad, where we observed that the routing direction alternates between neighboring layers and you may also have noticed that the wires have different dimensions on different layers. For instance, the wires on TM1 and TM2 are much thicker than on the layers below. Metal stacks are usually designed to have progressively larger wires, which has a reason.

Lower metal layers are primarily used for local connections both within standard cells (which are usually hidden EDA tools) and between neighboring cells. These layers have narrower wire and smaller spacing, providing higher routing resources to ensure the many connections between standard cells can be routed.

In contrast, upper metal layers are more used for global routing and the power grid. These layers have wider wires and larger spacing to reduce parasitics like resistance and capacitance, which is critical for long-distance connections and for carrying power and ground signals. Wider wires in the upper layers help minimize resistance, thereby reducing the IR-drop (voltage drop due to resistance in the power delivery network), which will be covered in the next exercise Exercise - Power analysis.

Student Task 1

We want to find out how many routing resources we have in the IHP130 technology that is available to us during the routing stage. There are multiple ways to get this information:

1. **The technology documentation**: This is where all the layout rules that are specified by the manufacturer. If your design does not fulfill those rules, the manufacturer will reject your design, since it cannot guarantee the correctness of the manufactured chip. The number of rules is usually very exhaustive, and it also includes a lot more layers than what we are exposed to during the backend design with OpenRoad. You can find the documentation `docs/SG13G2_layout_rules.pdf`. The section for `Metal1` for instance is 5.16, which defines the minimum spacing, and width from which you can derive the minimum pitch.
2. **The technology file**: The technology file is an LEF file that includes the design rules that are relevant for the EDA tool, such as OpenRoad. You can find the tech lef file under `technology/lef/sg13g2_tech.lef`. The technology LEF includes sections for every layer which include the pitch, width and spacing rules
3. **In the OpenROAD GUI**: If you have a design open in OpenRoad, you can select a layer in *Display Control*, and you will see its properties in the *Inspector*.

- By checking in the documetation, fill out the metal stack table below with the information about minimum width & spacing:

Layer Name	Direction	Width [nm]	Spacing [nm]	Pitch [nm]
`Metal1`	vertical			
`Metal2`	horizontal			
`Metal3`	vertical			
`Metal4`	horizontal			
`Metal5`	vertical			
`TopMetal1`	horizontal			
`TopMetal2`	vertical			

The *pitch*, which refers to the distance between the centers of two wires, is not a layout rule you will find in the documentation since it is already covered by the minimum width and spacing of two wires. The pitch refers to the distance between the centers of two wires, meaning the pitch is, in theory, just the sum of the spacing and width. However, the minimum spacing rule also depends on the width of the wires and the *parallel runlength*. The detail routing stage will use *Routing Tracks* to guide the routing engine where to place the wires. The pitch of these tracks is constant and in order to accommodate different wire widths and runnlengths the pitch might be chosen more conservatively than the minimum pitch.

- To complete the table with the wire pitch, check the technology LEF to find out which default pitch is used for each layer.

> ***Note:*** *The different spacings for different widths and parallel run lengths are defined in the* `SPACINGTABLE` *property. If you compare this to the layout rules in the documentation, you will find that this table represents all the spacing rules from the documentation.*

After filling out the table, you might also realize why the naming convention of the upper two layer differs. Those two layers have much thicker wires and they are mainly used for the powergrid instead of routing the signals. Therefore, we mainly consider `Metal1` to `Metal5` for signal routing.

- Calculate the combined routing resources of the signal routing layers in terms of `wires/mm` in the horizontal and vertical directions:

Direction	Layers	Routing resources [wires/mm]	over SRAM [wires/mm]
vertical	`Metal1`, `Metal3`, `Metal5`		
horizontal	`Metal2`, `Metal4`		

However, usually we don't have the entire routing resources available. For instance, SRAM macros typically obstruct the lower layers and only leave a few layers on top for routing. To find out which layers are still available, open the design in OpenROAD:

```
sh> cd openroad
sh> oseda -2025.01 openroad -gui
```

From the OpenROAD console we load the design after the floorplan stage:

```
OpenROAD > read_db save/croc.floorplan/croc.floorplan.odb
```

There are a couple of SRAM macros placed in the design, and we can display the routing blockages over them.

- Figure out which layers can be actually be used if we need to route over SRAMs.

> ***Hint:*** *It is easiest if you disable the* **Nets** *and all the* **Layers** *at first and then gradually enable layers one by one. Also, make sure that* **Blockages** *is enabled.*

Furthermore, not all the tracks can be used for signal routing. Some of them are also used for the power grid.

- Assuming the power grid occupies 20% of all routing tracks, what are the routing resources over the SRAMs in both directions? Complete the previous table.

Part 2: Routing Track

Before the actual routing process begins, the design must be prepared by defining **routing tracks**. Routing tracks serve as a predefined grid on each metal layer that guides the placement of wires. The router engine will later use these tracks to determine where interconnections can be placed.

Each track represents a potential wire position, and the track spacing is determined by the pitch of the metal layer. As we learned in the previous exercise, the pitch is determined by minimum width and spacing constraints, which vary across layers.

Student Task 2

The routing tracks are usually created after creating the floorplan and before constructing the powergrid. At this time, you should still have the design loaded after the floorplan stage. To create the tracks, you can use the following command:

```
OpenROAD > make_tracks
```

Now, you can inspect the routing tracks in the GUI. In *Display Control*, make sure that you enable the *Tracks*, and they will become visible when you zoom in closer. You will notice that OpenRoad creates tracks in both preferred and non-preferred direction.

Use the ruler to measure the routing track pitch of one of the layers, to confirm that it matches with the one from the previous exercise.

> ***Hint:*** *With* `K`*, you can enable the ruler, and you need to press the* `Ctrl` *key to prevent snapping, since the ruler does not recognize the tracks as objects it can snap to.*

The `make_tracks` command takes the routing pitches and offset of the tracks directly from the technology LEF file. However, you can also overwrite the default values with `-x/y_pitch` and `-x/y_offset` if needed.

Part 3: Global Routing

After defining routing tracks, the next step in the routing process is **global routing**, which provides a high-level plan for interconnects across the design. Instead of determining the exact wire placements, global routing divides the chip area into a coarse routing grid and assigns signal paths to regions known as **global routing cells (GCells)**. These cells represent small sections of the design where wires will eventually be placed during detailed routing. The goal of global routing is to estimate wire paths while considering congestion, layer assignment, and overall routing feasibility.

During this stage, the router calculates paths for each net while ensuring an even distribution of routing resources to prevent excessive congestion in certain regions. It also assigns nets to appropriate metal layers, ensuring that critical signals use layers with lower resistance and that vias are minimized to improve timing and reliability. Another key aspect of global routing is congestion analysis. The router estimates the demand for routing resources in different areas and adjusts paths accordingly to balance wire density, avoiding excessive detours or bottlenecks that could degrade performance.

Student Task 3

Hint: *Some of the commands need to be repeated multiple times throughout the exercise. We therefore recommend to collect them in a script that you can source again.*

Load the Design

In this task, we will run global routing. First, we need to load the design that we got after the last exercise i.e. the CTS stage:

```
OpenRoad> source scripts/init_tech.tcl
OpenRoad> read_db save/croc.cts/croc.cts.odb
OpenRoad> read_sdc save/croc.cts/croc.cts.sdc
```

Configure Routing Layers

Before running global routing, we need to define the metal layers that will be used for signal and clock routing. Use the following command to set the routing layers:

```
OpenRoad > set_routing_layers -signal Metal2-Metal5 -clock Metal2-Metal5
```

`Metal1` is not included because standard cells already use it for internal connections. To understand why `Metal1` is excluded, enable only `Metal1` in the GUI and observe how it is primarily occupied by local cell routing. Similarly, `Metal2` is sometimes used for standard cell pins, but only more complex cells rely on `Metal2` pins, where the `Metal1` routing resources do not suffice. Lastly, we set `Metal5` as the maximum routing layer since, as seen in previous exercises, the `TopMetal` layers are mainly reserved for power delivery rather than signal routing.

Optional: *From the GUI, find out which type of cells use `Metal2` pins.*

Run Global Routing

With the routing layers defined, execute the following command to perform global routing:

```
OpenRoad > global_route -congestion_report_file reports/congestion.rpt
```

This step generates an initial routing plan while producing a congestion report, which we will analyze in the GUI later. The command may take a minute to complete. In some cases, you might encounter an error such as:

```
[ERROR GRT-0119] Routing congestion too high. Check the congestion heatmap in the GUI.
```

If this happens, do not worry—we will investigate congestion issues next.

Hint: *If you run this in a script, you might want to enable the `-allow_congestion` flag, which resumes the script even if the `global_route` command throws some errors.*

Visualizing the GCell Grid

The GCell grid is the coarse routing grid that global routing uses to divide the chip area into manageable sections. To view this grid in the GUI, go to: `Display Control → Misc → GCell Grid`

This grid is essential for global routing, as each GCell represents a region where wires are assigned before detailed routing takes place.

Analyzing Routing Congestion

Now, we will examine the routing congestion heatmap using the congestion report we generated earlier. To display it in the GUI, go to: `Heat Maps → Routing Congestion`

Each `GCell` is color-coded based on the number of wires passing through it. Areas with high congestion are marked in red, indicating potential routing bottlenecks. If you encountered an error during global routing, you will likely see some red GCells, highlighting the most congested regions.

It is important to note that congestion in global routing does not necessarily mean detailed routing will fail. However, excessive congestion can lead to numerous Design Rule Checking (DRC) violations in the next stage. If detailed routing produces too many violations on a particular layer, we may need to revisit the global routing stage and adjust layer usage.

> ***Hint:*** *OpenROAD provides a mechanism to adjust congestion during global routing with the* `set_global_routing_layer_adjustment` *command that artificially reduces the number of tracks that is available to the global router.*

Part 4: Optimize Timing

At this stage of the flow, we can estimate parasitics to gain an early insight into the impact of interconnect resistance and capacitance on timing. Since global routing has already assigned approximate wire paths and routing layers, we can use this information to model RC parasitics more accurately. This estimation helps in identifying potential timing violations before proceeding to detailed routing, allowing for adjustments in layer assignment or congestion handling if necessary.

Student Task 4

First, update the timing report with `Timing Report > Update`. By checking the timing report, do you have any setup or hold violations? Note down the worst negative slack for both setup and hold.

We can estimate the parasitics after global routing to have a more accurate view of the timing which also considers the delay of the wires.

```
OpenRoad> estimate_parasitics -global_routing
```

After that, update the timing again and check the `Timing Report`.

How did the timing change? Did the worst negative slack for setup/hold get worse or better? What's the reason?

The timing with estimated parasitics should have got worse, since we take into account the interconnect resistance and capacitance now. In our case the slack is now negative, meaning we don't meet the timing anymore. We can try to fix the setup timing violations with:

```
OpenRoad> repair_timing -setup -repair_tns 100
```

The `-repair_tns` argument just tells the tool that it should try to fix **all** (i.e. 100%) of the violating paths. Without specifying this, the tool would just try to fix the worst one.

Update and check the timing report again to check if the setup violations were fixed now. Ideally, your slack should again be positive, meaning we meet the timing again. How did the tool manage to fix the setup violations?

> ***Hint:*** *The output of the* `repair_timing` *command should give you some hints.*

Since we are at it, we can also fix the hold violations:

```
OpenRoad> repair_timing -hold -hold_margin 0.05 -repair_tns 100
```

What happened during hold timing fixing? i.e. what's the difference between setup and hold timing fixing?

For both setup and hold fixing, we can also provide a margin with `-hold/setup_margin` argument. The tool will then additionally try to achieve a WNS that is larger than the margin that we specified. This is especially useful for hold violations since they can break the entire chip (setup violations will just lower your maximum frequency).

If both setup and hold timing violations are now repaired, we face another problem, that is identified by running the standard cell placement check again:

```
OpenRoad> check_placement -verbose
```

Can you identify the problem?

During setup and hold fixing, the tool will resize the gates and also insert buffers wherever needed. However, it does not *legalize* the placement of the new or resized cells, meaning they can now overlap now and and need to be *legalized* again. For instance, have a look at one of the cells that have a reported overlap:

```
OpenRoad> select -type Inst -name cell_name
```

Where `cell_name` is one of the cells from log output of `check_placement`. After selecting the cell, you can press the `Zoom to` button to display the cell, where you can see that the cells now overlap.

To fix this, we can do the following:

```
OpenRoad> global_route -start_incremental
OpenRoad> detailed_placement
OpenRoad> global_route -end_incremental
```

The first command, tells the global router to start recording all the nets that are modified. Then `detailed_placement` will *legalize* the placement of the cells, such that they won't overlap anymore. Finally, the second `global_route -end_incremental` command will perform global route again, but only on the nets that were modified during the detail placement.

After this, check again that all cells are now *legalized*:

```
OpenRoad> check_placement
```

And check that timing can be met now. Note that you should run the parasitic extraction again before updating the timing, since we changed the global routes:

```
OpenRoad> estimate_parasitics -global_routing
```

Part 5: Process Antennas

The *antenna effect*, also known as plasma-induced damage, is a phenomenon that occurs during the fabrication of integrated circuits, particularly during the metal etching process. It arises due to the accumulation of charge on floating metal interconnects before they are connected to a diffusion region (such as a transistor gate).

During plasma etching, high-energy ions remove unwanted material, but this process can cause electrons to accumulate on isolated metal structures. If a metal wire or via has a large surface area but is not yet connected to a discharge path, the accumulated charge can induce a voltage spike. This excessive voltage can damage the thin gate oxide of transistors, leading to device degradation or failure. For instance, if you look at Figure 1 below, the area of the *Antenna net* is too large compared to the area of the gate.

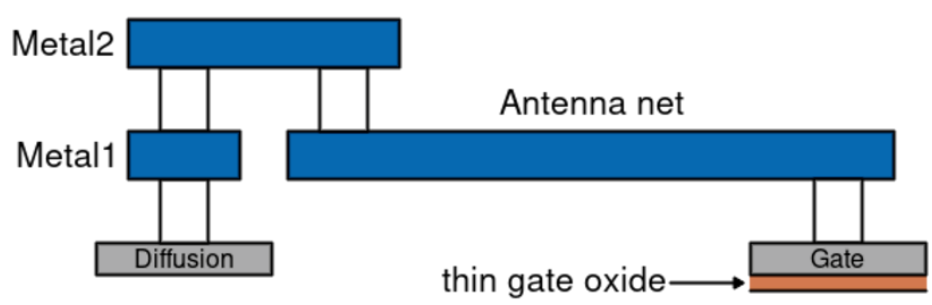

Figure 1: Antenna effect during manufacturing

In general, there are two solutions to solve this problem. The first is inserting antenna diodes. These are specially designed diodes placed near transistor gates to provide a safe discharge path for accumulated charge during the fabrication process. By connecting the floating metal segment to the diode, excess charge is redirected to the substrate, preventing damage to the thin gate oxide. An example of this is shown in Figure 2.

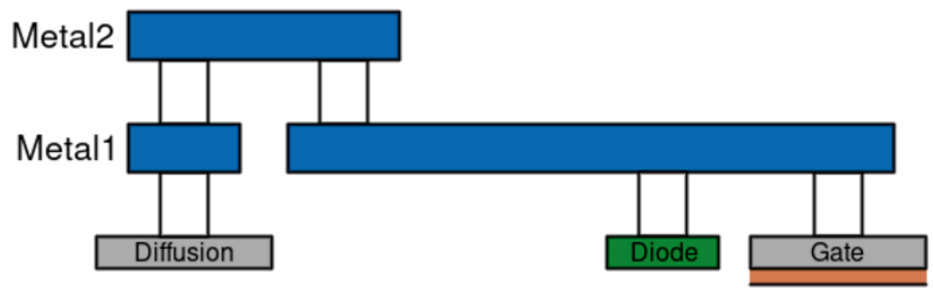

Figure 2: Diode insertion

Student Task 5

In OpenRoad, we can check if there are any antenna violations with the following command:

```
OpenRoad> check_antennas
```

and subsequently, OpenRoad can fix antenna violations with the following command:

```
OpenRoad> repair_antennas
```

OpenRoad will also report how many diodes it had to insert to fix all antenna violations. You can also have a look at one of the inserted diodes:

```
OpenRoad> select -type Inst -name ANTENNA_1
```

Where `ANTENNA_1` is just the name of the first diode that was inserted. If you zoom in, you might be able to guess to which cell it belongs to since the diodes are usually inserted as close to the violating cell (respectively pin) as possible. This will be more clear after we finished the detail route stage in the next part.

The other solution is to perform *layer hopping* which is shown in Figure 3. This essentially breaks up long wires segments by hopping to higher layers. In the example shown below, the long wire segment should not cause any problems anymore, since during the fabrication of the `Metal1`, there is no direct path to the gate anymore, so the thin gate oxide is not damaged during the fabrication process. Layer hopping is usually the preferred solution, but is currently not supported by OpenRoad and would need to be done by hand.

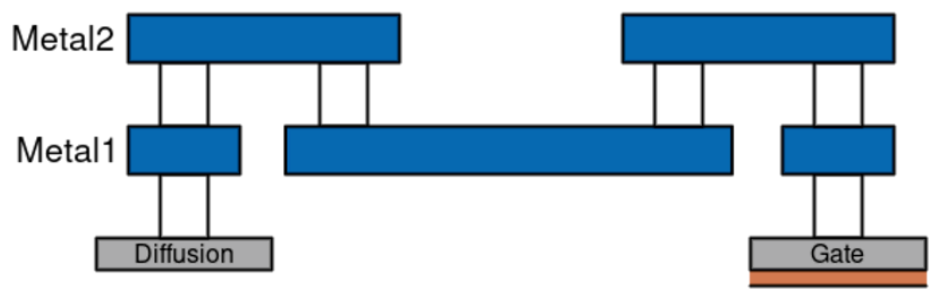

Figure 3: Layer Hopping

Part 6: Detail Routing

After global routing has provided an approximate path for interconnects, the design moves to the detailed routing stage. In this step, the router determines the precise wire placements, ensuring that all nets are fully connected while adhering to design rules such as minimum spacing, via constraints, and layer-specific restrictions. Unlike global routing, which operates on a coarse grid, detailed routing works at the track level, assigning exact locations for each wire.

Student Task 6

Running the detail route stage in OpenRoad essentially consists of a single command `detailed_route`, but first we want to setup a multithreading since the routing stage can take quite long (in the order of minutes for our design). Luckily, routing is a parallelizable algorithm, and we can set up the number of threads in OpenRoad with:

```
OpenRoad> set_thread_count 6
```

Then we can run the detail routing in OpenRoad with the following command. Note that this command will take in the order of 15-20 minutes. You can also speed it up by lowering the number of iterations at the expense of more routing violations.

```
OpenRoad> detailed_route -output_drc reports/croc_route_drc.rpt \
             -bottom_routing_layer Metal2 \
             -top_routing_layer Metal5 \
             -droute_end_iter 10 \
             -clean_patches \
             -verbose 1
```

The detail route is an iterative task, and in every iteration, it checks the DRC violations, which it then tries to fix in this iteration. You will see that in the beginning, it will report thousands of DRC violations, but with every iteration it should reduce. The detailed routing algorithm will stop if it manages to fix all the DRCs, or if it reaches the specified maximum number of iterations, provided with the `-droute_end_iter` flag. In our case, 10 iterations should be enough to fix all the violations. In more complex designs, you might need more iterations, but it is also possible that the tool will not find a solution and the number of violations remains the same. You can check in the logs whether the number of violations converges to zero. If the tool does not manage to fix the violations, there might be a multitude of reasons why the tool does not manage to fix the violations. Some of them also require you to revisit previous stages like floorplanning, powerplanning and placement.

To analyze the remaining DRCs in the GUI you can open the DRC reports with `Windows > DRV Viewer`, and you can load the report under `reports/croc_route_drc.rpt`.

Part 7: Filler Cells

After detailed routing, *filler cells* are inserted to maintain power and ground rail continuity in the design. Achieving 100% utilization during placement is practically impossible due to the many degrees of freedom involved. Factors such as varying cell sizes, spacing constraints between neighboring cells, and the need for late-stage cell insertions—such as buffers for hold-time fixing—make it infeasible for the placement tool to completely fill all available space. As a result, small gaps inevitably remain, which must be filled to ensure an uninterrupted power grid. Filler cells do not contain active logic but extend the power and ground connections.

Student Task 7

The command in OpenRoad to run the filler insertion is:

```
OpenRoad> filler_placement {sg13g2_fill_8 sg13g2_fill_4 sg13g2_fill_2 sg13g2_fill_1}
```

Note that there are multiple filler cells the tool can choose from. The only difference is in the width of the cell. By default, the tool will try to use the largest filler cells available if it fits into the free space. After inserting the filler cells, have a look at the design. You should see that there are no available spaces left in the std cells rows.

Finally, we connect all the standard cells to the power and ground nets:

```
OpenRoad> global_connect
```

Note that this connects the net only *logically* to the nets, which means that the modifications are only performed in the netlist. The physical connection of the power and ground nets is already ensured with the insertion of the filler cells. The logical connection is required to prevent any mismatches of the resulting netlist and the actual layout in LVS, which will be the topic of the next exercise.

Part 8: Quality of Result (QoR)

The final step in the physical design flow is to analyze the QoR of the implementation and prepare for potential refinements in the next iteration. While chip area is the primary cost driver in IC design and was already determined during floorplanning, analyzing the final design's timing, routing, and utilization can reveal opportunities for improvement. These insights help determine whether the area should be expanded or

squeezed in future runs. Since physical design is inherently iterative, understanding these factors will allow you to refine the flow for better results. The following points will guide you in reflecting on the final design and considering potential adjustments for a future rerun, helping you optimize your own tapeout process.

Timing

The most critical performance metric is whether the design meets setup and hold timing constraints. Check the timing reports and reflect on the following:

- Does the design meet both setup and hold timing?
- What is the achievable clock frequency? Does it meet your target or expected frequency?
- If not, identify the longest path in the design. What are the possible ways to fix the critical path?
  - Consider whether changes should be made at the RTL level or during synthesis and/or backend.
  - Would modifications require a full rerun, or could they be addressed with minor changes?
  - How much time do you have left until the tapeout deadline? Can you do a full rerun or only minor changes?

Routing Violations

Routing issues can impact manufacturability and require review before finalizing the design. To analyze the routing, answer the following:

- How many design rule violations (DRCs) remain after detailed routing?
  - Can you identify the type of routing violations that you have?
  - Are there any systematic issues like severe routing congestion in your design? Do you need a larger floorplan?
  - Is the number of violations low enough that you could fix the remaining ones by hand?

Area and Cost

Since the area directly affects manufacturing costs, evaluating utilization is key for future iterations. Even though the area was set during floorplanning, the final layout may reveal opportunities for improvement. Consider:

- Are there areas in the design that appear underutilized?
- Could the area of the chip be reduced while still maintaining routability?
- On the other hand, if routing congestion or placement was a major issue, should the floorplan be expanded slightly to provide more space?

Part 9: Output

Once the design is finalized and meets all constraints and our expectations, the last step is to generate the necessary output files for verification, signoff, and further processing.

Student Task 8

DEF Layout file

The DEF file contains the physical layout information of the design, including cell placement, routing, and floorplanning. This will be used in the next exercise to generate the actual GDS layout file that is used for LVS/DRC checking.

```
OpenRoad> write_def out/croc.def
```

Verilog Netlist

We write out two different netlists:

1. The normal netlist with the additional cells that we inserted during the backend process (e.g. buffers). This netlist can be used for post-layout simulations.
2. The LVS netlist, which additionally includes all power (VDD) and ground (VSS) connections. This netlist is used for the LVS step that is done in the next exercise.

```
OpenRoad > write_verilog out/croc.v
OpenRoad > write_verilog -include_pwr_gnd out/croc_lvs.v
```

SDC Constraints

The SDC file contains the timing constraints used during the design process, including clock definitions, timing exceptions (e.g., false paths), and delay. It can be used for further (and more accurate) *signoff verification* in other tools using Static Timing Analysis (STA).

```
OpenRoad > write_sdc out/croc.sdc
```

ODB Database

The ODB is an internal database file that stores the complete state of the design, including placement, routing, and constraints. It allows OpenROAD to reload the design for further iterations or modifications without reprocessing all previous steps. We can also easily regenerate the above files from the database again.

```
OpenRoad > write_db out/croc.odb
```